Initial VVA project setup

Repository: applet | **Branch:** master | **Ref:** master | **Date:**

1. Executive Summary & Purpose

This commit establishes the foundational structure for the VVA (Version & Workflow Automator) project. It adds a comprehensive set of source files, configuration files, and dependencies required to run the application locally and to orchestrate the Git■to■AI■to■Document pipeline.

Purpose and Strategic Rationale:

The purpose of this commit is to bootstrap the VVA codebase, providing the core CLI, React UI, Git data collection logic, AI analysis engine, and document generation utilities. It also includes environment configuration, package manifests, and lock files to ensure reproducible builds.

2. System Architecture & Component Workflow

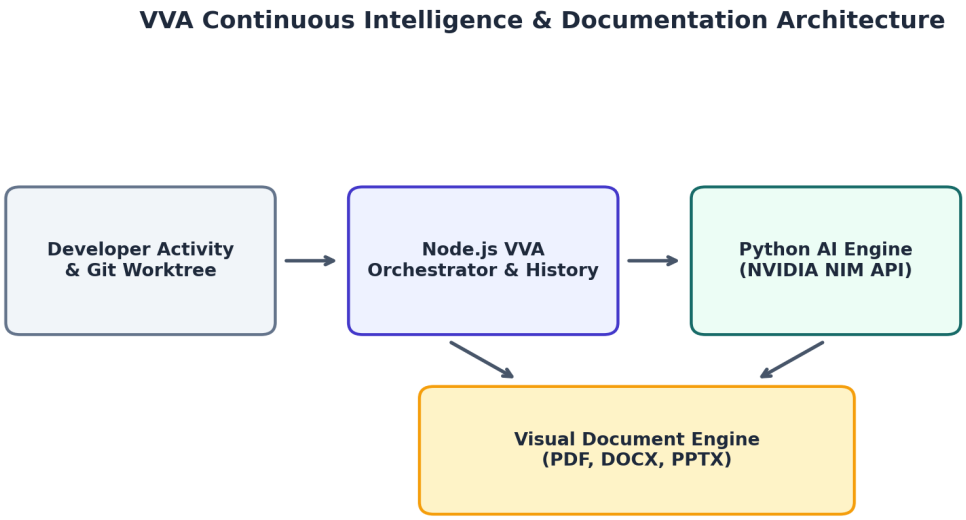


Figure 1: Architectural Component Flow & Intelligence Lifecycle

3. Development Activity & Evolution Metrics

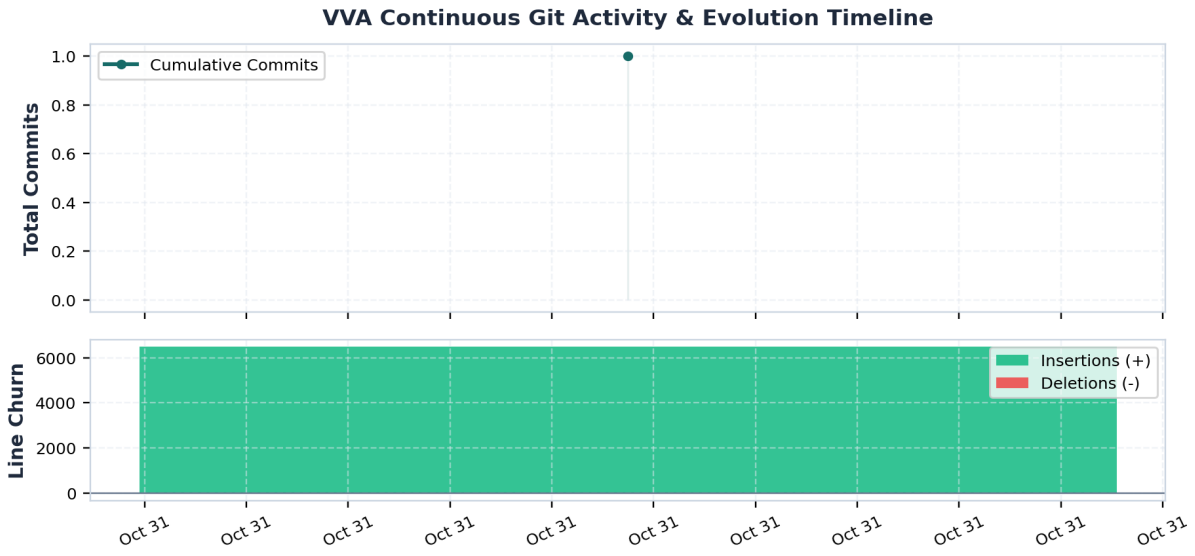


Figure 2: Cumulative Commit Progress & Code Volume Churn

4. Historical Development Milestones

Commit	Author	Milestone Type	Commit Message
9782073	Abhiram Binil	Project Inception & Setup	feat(vva): initial commit of Version & Workflow Automator

5. Technical Implementation & Subsystem Modifications

- Added environment example with NVIDIA API key and model configuration
- Created .gitignore to exclude build artifacts and logs
- Provided README with local run instructions
- Implemented ai/ai_engine.py to invoke NVIDIA API for commit analysis
- Added ai/test_nvidia.py for API connectivity test
- Created bin/vva.js as CLI entry point
- Generated bun.lock with full dependency tree
- Set up index.html and metadata.json for web deployment
- Included comprehensive project description in public/assets
- Defined JSON schemas for AI response and project data
- Built React application entry point (src/App.tsx) and main.tsx
- Implemented CLI command modules (init, report, status, watch)
- Implemented core Git utilities for repository detection and data extraction
- Implemented core collector for gathering commit metadata and diffs
- Implemented core Python module for invoking AI and document engines

- Created UI components for terminal, collector, AI engine, document generation, architecture, team, and header
- Added utility functions for orchestrating data flow and generating structured AI responses
- Added TypeScript types for VVA data structures
- Added PDF and Word export utilities
- Configured TypeScript compiler options and Vite build settings

6. Architectural Impact & Engineering Observations

The commit establishes a layered architecture: a Node.js orchestrator (CLI & React UI) coordinates Git data collection, AI analysis, and document generation. The AI engine runs in Python, communicating via JSON files. The project is modularized into commands, core utilities, components, and schemas, enabling clear separation of concerns and facilitating future extensions.

Observations and Technical Limitations:

- The commit only adds files; no functional tests or CI configuration are present.
- The AI engine script relies on an NVIDIA API key hard■coded in .env.example, which may not be secure for production.
- The bun.lock file is extremely large, potentially impacting install times.
- The project includes both Node.js and Python components but does not provide a unified build or deployment pipeline.
- No error handling for missing dependencies or environment variables is included in the CLI commands.
- The React UI references assets that may not be present in the repository root.
- The commit does not include any documentation for the API endpoints used by the AI engine.
- The TypeScript configuration uses experimental decorators without corresponding code.
- The project relies on a specific version of the NVIDIA model (meta/llama-3.1-70b-instruct) but does not provide fallback logic.